Practical Exercise

Google BigQuery

Dataset: Retail Sales Dataset

Q1. Filter all transactions that occurred in the year 2023.

```
---Question1---

SELECT *
FROM `august-ensign-493417-h3.70177.RetailSales`
WHERE EXTRACT(YEAR FROM `DATE`) = 2023;
```

Query results

+ Create conversation Save results Open in

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity
1	191	2023-10-18	CUST191	Male	64	Beauty	
2	204	2023-09-28	CUST204	Male	39	Beauty	
3	230	2023-04-23	CUST230	Male	54	Beauty	
4	232	2023-02-06	CUST232	Female	43	Beauty	
5	309	2023-12-23	CUST309	Female	26	Beauty	
6	310	2023-10-12	CUST310	Female	28	Beauty	

Q2. Display all transactions where the Total Amount is more than the average Total Amount of the entire dataset.

```sql
----Question2----


SELECT *
FROM `august-ensign-493417-h3.70177.RetailSales`
WHERE `Total Amount` > (
    SELECT AVG(`Total Amount`)
    FROM `august-ensign-493417-h3.70177.RetailSales`
);
```

Query results

+ Create conversation | Save results | Open in

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity
1	21	2023-01-14	CUST021	Female	50	Beauty	
2	28	2023-04-23	CUST028	Female	43	Beauty	
3	128	2023-07-05	CUST128	Male	25	Beauty	
4	220	2023-03-03	CUST220	Male	64	Beauty	
5	238	2023-01-17	CUST238	Female	39	Beauty	
6	364	2023-08-23	CUST364	Female	19	Beauty	

Q3. Calculate the total revenue (sum of Total Amount).

```sql
---Question3---

SELECT SUM(`Total Amount`) AS Total_Revenue
FROM `august-ensign-493417-h3.70177.RetailSales`;
```

Query results

+ Create conversation | Save results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Total_Revenue
1	456000

Q4. Display all distinct Product Categories in the dataset.

```
---Question4----

SELECT DISTINCT `Product Category`
FROM `august-ensign-493417-h3.70177.RetailSales`;
```

Query results

+ Create conversati

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Product Category
1	Beauty
2	Clothing
3	Electronics

Q5. For each Product Category, calculate the total quantity sold.

```
---Question5--

SELECT
    `Product Category`,
    SUM(Quantity) AS Total_Quantity
FROM `august-ensign-493417-h3.70177.RetailSales`
GROUP BY `Product Category`;
```

Query results

+ Create conversation | Save results | Open in

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Product Category	Total_Quantity
1	Beauty	771
2	Clothing	894
3	Electronics	849

Q6. Create a column called Age_Group that classifies customers as: – 'Youth' (<30) lt' (30–59) – 'Senior' (60+)

```sql
---Question6--

SELECT
    `Customer ID`,
    Age,
    CASE
        WHEN Age < 30 THEN 'Youth'
        WHEN Age BETWEEN 30 AND 59 THEN 'Adult'
        WHEN Age >= 60 THEN 'Senior'
    END AS Age_Group
FROM `august-ensign-493417-h3.70177.RetailSales`;
```

Query results

+ Create conversation Save results Open in

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Customer ID	Age	Age_Group
1	CUST191	64	Senior
2	CUST204	39	Adult
3	CUST230	54	Adult
4	CUST232	43	Adult
5	CUST309	26	Youth
6	CUST310	28	Youth
7	CUST363	64	Senior

Q7. For each Gender, count how many high-value transactions occurred (where Total Amount > 500). Expected Output: Gender, High_Value_Transactions

```sql
--Question7--

SELECT
    Gender,
    COUNTIF(`Total Amount` > 1000) AS High_Value_Transactions
FROM `august-ensign-493417-h3.70177.RetailSales`
GROUP BY Gender;
```

Query results

+ Create conversation Save results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Gender	High_Value_Tran...
1	Male	77
2	Female	76

Q8. For each Product Category, show only those categories where the total revenue exceeds 5,000.

```sql
--Question8--

SELECT
    `Product Category`,
    SUM(`Total Amount`) AS Total_Revenue
FROM `august-ensign-493417-h3.70177.RetailSales`
GROUP BY `Product Category`
HAVING SUM(`Total Amount`) > 50000;
```

Query results

+ Create conversation Save results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Product Category	Total_Revenue
1	Beauty	143515
2	Clothing	155580
3	Electronics	156905

9. Display a new column called Unit_Cost_Category that labels a transaction as: – 'Cheap' if Price per Unit < 50 – 'Moderate' if Price per Unit between 50 and 200 – 'Expensive' if Price per Unit > 200

```sql
---Question9--

SELECT
    `Transaction ID`,
    `Price per Unit`,
    CASE
        WHEN `Price per Unit` < 50 THEN 'Cheap'
        WHEN `Price per Unit` BETWEEN 50 AND 200 THEN 'Moderate'
        WHEN `Price per Unit` > 200 THEN 'Expensive'
    END AS Unit_Cost_Category
FROM `august-ensign-493417-h3.70177.RetailSales`;
```

Query results

+ Create conversation Save results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Transaction ID	Price per Unit	Unit_Cost_Category
1	191	25	Cheap
2	204	25	Cheap
3	230	25	Cheap
4	232	25	Cheap
5	309	25	Cheap
6	310	25	Cheap
7	363	25	Cheap

Q10. Display all transactions from customers aged 40 or older and add a column Spending_Level showing: – 'High' if Total Amount > 1000 – 'Low' otherwise

```sql
--Question10---
SELECT
    `Customer ID`,
    Age,
    `Total Amount`,
    CASE
        WHEN `Total Amount` > 1000 THEN 'High'
        ELSE 'Low'
    END AS Spending_Level
```

Query results

+ Create conversation Save results Open in

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	Customer ID	Age	Total Amount	Spending_Level
1	CUST191	64	25	Low
2	CUST204	39	25	Low
3	CUST230	54	25	Low
4	CUST232	43	25	Low
5	CUST309	26	25	Low
6	CUST310	28	25	Low